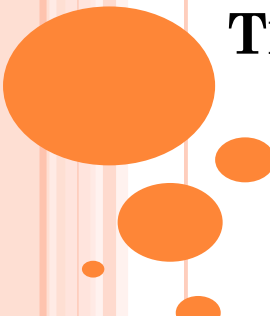


Arquitectura e Ingeniería de Computadores

Grado de Ingeniería del Software

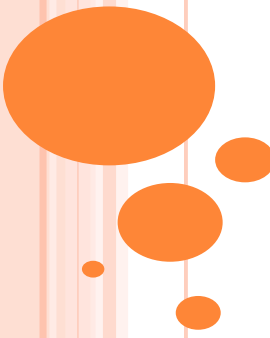


TEMA 4. JERARQUÍA DE MEMORIA (PARTE 1)

Óscar David Robles Sánchez
Luis Rincón Córcoles
David Miraut Andrés
Ángel Serrano Sánchez de León

ÍNDICE

- Introducción a la jerarquía de memoria
 - Introducción
 - Tecnologías de memoria
 - Parámetros del rendimiento
- Memoria caché
 - Introducción
 - Elementos de diseño
 - Medición del rendimiento



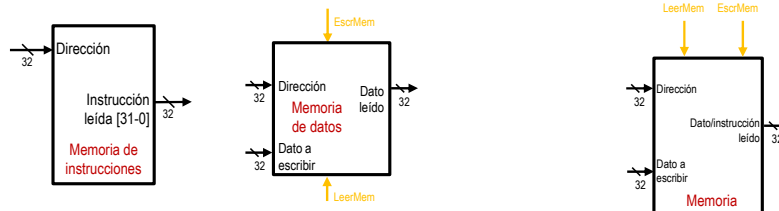
INTRODUCCIÓN A LA JERARQUÍA DE MEMORIA

INTRODUCCIÓN

- Computadores primigenios: programas cableados.
- **Computador de programa almacenado** (Zuse, 1936): dispone de una memoria que almacena el programa sin necesidad de reestructurar cables.
- **Memoria principal:**
 - Unidad funcional del computador en la que se almacenan las instrucciones y datos que componen el programa que se encuentra en ejecución.

- En los computadores anteriores a 1936, los programas se introducían en el ordenador mediante la configuración de un conjunto de cables, lo cual hacía pesada la programación.
- Concepto de computador de programa almacenado (Konrad Zuse, 1936): la memoria principal almacena tanto los datos como los programas, sin necesidad de cambiar la circuitería.
- Hoy en día hablamos de memoria principal como aquella unidad funcional del computador que almacena tanto los programas en ejecución como los datos en uso.

INTRODUCCIÓN



Arquitectura Harvard:

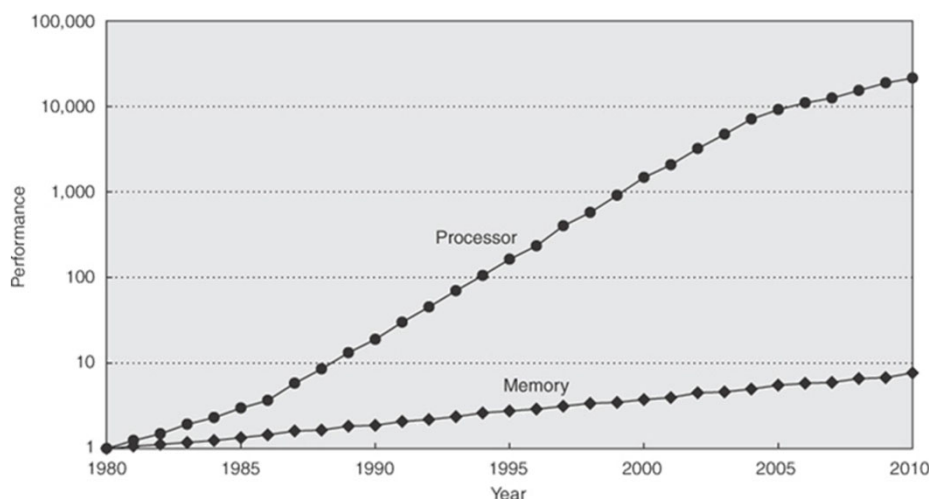
- Memorias de datos e instrucciones separadas

Arquitectura de von Neumann:

- Memoria unificada para datos e instrucciones

- Según si disponemos de memorias unificadas o separadas para los datos y las instrucciones, las arquitecturas de computadores se clasifican en:
 - Arquitectura Harvard:
 - Memorias separadas, una para albergar los programas y otra para almacenar los datos de dichos programas.
 - Es la usada en la implementación uniciclo de MIPS.
 - Arquitectura de von Neumann:
 - Memoria unificada, que contiene tanto los programas como sus datos.
 - Es la configuración habitual hoy en día.
 - Es la usada en la implementación multiciclo de MIPS.

INTRODUCCIÓN



Ley de Moore y el salto memoria-procesador

6

- Esta gráfica muestra la ley de Moore aplicada a los procesadores y a las memorias.
 - En el eje X se muestra el año, mientras que el eje Y está el rendimiento en escala logarítmica.
 - Por tanto, lo que parece una recta ascendente, en realidad es un crecimiento exponencial.
 - Los datos de los procesadores representan su capacidad de cómputo, o el número de transistores que se utilizan en su construcción, mientras que en los datos de memoria se indica su ancho de banda o la latencia.
 - Normalizando los datos a 1 para el año 1980, vemos cómo los procesadores han aumentado su rendimiento de manera muchísimo mayor a como lo han hecho las memorias.
- Ley de Moore de los procesadores:
 - Publicada por Gordon Moore en 1965 (quien fundaría la empresa Intel tres años después).
 - Es la observación empírica de que el número de transistores en un procesador, y con ello las capacidades o la medida de velocidad del mismo, se duplica aproximadamente cada 18-24 meses.
 - Se trata de una ley experimental, no una ley física de la naturaleza.
 - En el rango de 30 años que muestra la figura el rendimiento del procesador se ha duplicado unas quince veces, lo cual significa que ha seguido un crecimiento exponencial desde 1 hasta $2^{15} = 32000$ aproximadamente.
- Comportamiento de la memoria:
 - No ha sido capaz de seguir el mismo ritmo que los procesadores.
 - La memoria de referencia que se utiliza en el año 1980 es una memoria DRAM de 64 kb con un tiempo de ciclo de 250 ns.
 - Por comparación, la memoria típica de 2010 es de clase DDR3, con capacidad para 4 Gb, y un tiempo de ciclo de 37 ns (unas 7 veces más rápida).
 - Mejora en la latencia de la memoria:
 - Ritmo continuado de mejora de un 6 % anual (la latencia de la memoria de un año es 0,94 veces la latencia de la memoria del año anterior).
 - Efectivamente: $250 \text{ ns} \times 0,94^{30} = 39 \text{ ns}$, prácticamente el valor indicado de 37 ns.
 - Respecto de la capacidad de memoria DRAM:
 - Hasta 1996 se observa que se cuadruplica cada 3 años (incremento anual del 60 %).
 - Esta tasa de crecimiento se ha reducido últimamente.
 - En la década actual de 2010 se observa una duplicación de la capacidad cada 2-3 años (incremento anual entre 25-40 %).

Figura y datos extraídos del libro Hennessy & Patterson, Computer Architecture, a Quantitative Approach, 5.ª edición, págs. 73 y 99, 2012.

INTRODUCCIÓN

- La memoria nunca alcanza las velocidades del procesador, con lo que le resulta un lastre.
- A lo largo de los años, se han introducido otros componentes de memoria, organizados en una **jerarquía**, para intentar paliar la diferencia de velocidades:
 - Registros
 - Memoria caché
 - Memoria masiva

7

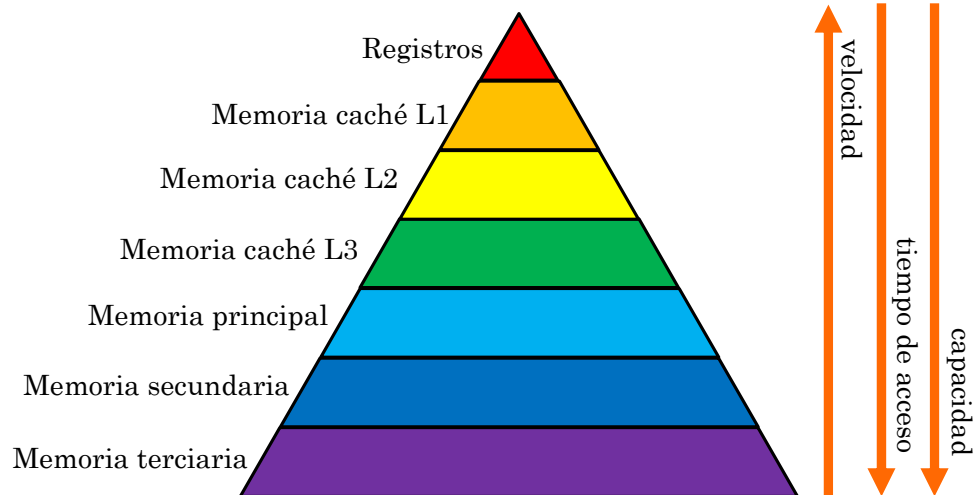
- Acceder a un dato en la memoria principal siempre es mucho más lento que cualquier operación realizada por el procesador.
 - La memoria supone un lastre que va frenando siempre al procesador.
- Los computadores disponen de otros componentes de almacenamiento, además de la memoria principal, de manera que se organizan en una jerarquía de memoria.
 - De mayor a menor velocidad (de menor a mayor capacidad), los componentes de la jerarquía de memoria son:
 - **Registros:**
 - Guardan datos temporales o resultados de cálculos intermedios recientes.
 - Son los más cercanos al procesador, por tanto, de acceso muy rápido (misma velocidad que el procesador).
 - A cambio son de muy baja capacidad (pocos bits).
 - Los registros están ubicados dentro del chip del procesador.
 - **Memoria caché:**
 - Memoria de acceso extremadamente rápido, pero bastante pequeña (decenas de kB a decenas de MB).
 - Contiene, a modo de recordatorio, los últimos datos o instrucciones consultados de la memoria principal, a los que puede volver a acceder a mayor velocidad.
 - Se suele dividir a su vez en varios niveles (3, por ejemplo).
 - La caché está ubicada dentro del chip del procesador.
 - **Memoria principal:**
 - Es más lenta que la memoria caché, pero de mayor capacidad (decenas de GB).
 - Contiene los programas en ejecución y los datos en uso.
 - Está ubicada en tarjetas de memoria que están conectadas en diversos zócalos de la placa base.
 - **Memoria masiva:**
 - Muy lenta, pero de gran capacidad (decenas de TB).
 - Almacena los programas y los datos que no estamos usando en este momento.
 - Tipos de memoria masiva:
 - Memoria secundaria:
 - Discos duros (magnéticos o de estado sólido).
 - Memoria terciaria (extraíble):
 - Periféricos de almacenamiento externo (CDROM, DVD, discos Blu-Ray, memorias flash, etc.).

INTRODUCCIÓN

- **Localidad temporal:** Si se accede a un dato, seguramente se volverá a acceder a él.
- **Localidad espacial:** Si se accede a un dato, seguramente se accederá a otros cercanos a él.
- La localidad en los programas surge de su estructura:
 - Secuencialidad
 - Bucles
 - Estructuras de datos

- La jerarquía de memoria se fundamenta en el principio de localidad:
 - En un momento concreto los programas acceden a una parte relativamente pequeña de su espacio de direcciones
- La localidad puede ser de dos tipos:
 - Temporal: Se suele acceder al mismo dato en instantes de tiempo diferentes.
 - Espacial: Se suele acceder a los datos vecinos de uno dado.
- La localidad se justifica por:
 - Secuencialidad:
 - Normalmente se accede a las instrucciones de los programas secuencialmente, es decir, unas detrás de otras.
 - Localidad espacial.
 - Bucles:
 - Se accede repetidamente a instrucciones y a datos.
 - Localidad temporal.
 - Estructura de datos (como arrays):
 - Los datos de la misma estructura se almacenan en posiciones consecutivas de memoria.
 - Localidad espacial.

INTRODUCCIÓN



9

- La jerarquía de memoria de un computador moderno se organiza en los siguientes niveles:
 - Registros, agrupados en un banco y de propósito general para la programación.
 - La memoria caché, normalmente organizada en 3 niveles (*levels*).
 - La memoria principal.
 - La memoria secundaria (disco duro).
 - La memoria terciaria.
- La velocidad aumenta hacia los niveles superiores de la jerarquía.
- El tiempo de acceso y la capacidad aumentan hacia los niveles inferiores de la jerarquía.

INTRODUCCIÓN

- Cada vez que el procesador necesita un dato, lo busca en el nivel superior de la jerarquía.
 - Si lo encuentra allí: **acierto** (*hit*).
 - Si no: **fallo** (*miss*).
- En caso de fallo, el procesador busca el dato en el nivel inmediatamente inferior de la jerarquía.
 - El dato es copiado en el nivel superior (localidad temporal).
 - Los datos vecinos son también copiados en el nivel superior (localidad espacial).
- La transferencia de datos entre niveles contiguos se realiza por **bloques**.

10

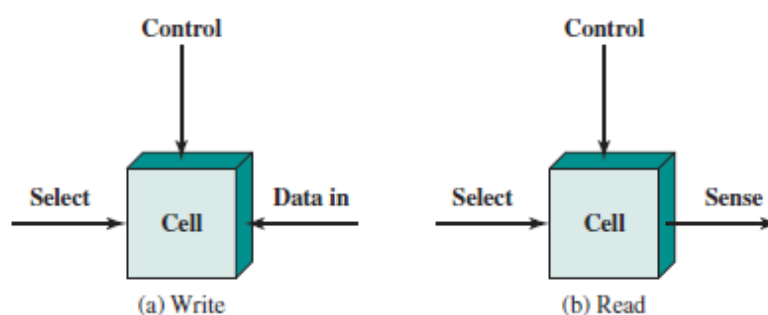
- La jerarquía de memoria funciona porque el procesador busca los datos que necesita en el nivel superior, que es muy rápido.
- Supongamos por simplicidad una jerarquía de memoria formada por 2 niveles:
 - Si lo encontramos en el nivel superior, se denomina acierto (accedemos al dato muy rápido).
 - Se llama tiempo de acierto (T_{acierto}) al tiempo necesario para buscar el dato deseado en el nivel superior.
 - Si contabilizamos cuántas veces se produce un acierto en un nivel de la jerarquía respecto al total de veces que accedemos a dicho nivel, obtenemos la tasa de aciertos, que suele expresarse en porcentaje ($\%_{\text{aciertos}}$).
 - Obviamente: $\%_{\text{fallos}} = 100 - \%_{\text{aciertos}}$.
 - El tiempo total invertido en caso de acierto es igual a T_{acierto} .
 - Si el dato buscado no está en el nivel superior, se llama fallo.
 - Hemos consumido un tiempo igual a T_{acierto} buscándolo infructuosamente en el nivel superior.
 - Hay que seguir buscándolo entonces en el nivel inferior (que es más lento), aunque no solo buscamos el dato, sino el resto de los datos vecinos con los que comparte bloque.
 - Futuros accesos a ese dato o a cualquiera de sus vecinos se resuelve con un acierto (acceso rápido).
 - El tiempo necesario para leer el bloque en el nivel inferior y copiarlo en el nivel superior se llama tiempo de penalización por fallo ($T_{\text{penalización}}$).
 - El tiempo total invertido en caso de fallo es igual a $T_{\text{acierto}} + T_{\text{penalización}}$.
- La comunicación entre dos niveles contiguos de la jerarquía se realiza por bloques de varias palabras (unidad de transferencia).
 - El tamaño de los bloques puede ser diferente según el “piso” de la jerarquía en el que nos encontremos.

ÍNDICE

- Introducción a la jerarquía de memoria
 - Introducción
 - **Tecnologías de memoria**
 - Parámetros del rendimiento
- Memoria caché
 - Introducción
 - Elementos de diseño
 - Medición del rendimiento

TECNOLOGÍAS DE MEMORIA

- Tecnología de **semiconductores** (transistores).
- **Celda de memoria** = 1 bit.
- Direcciones de memoria.
- Unidad direccionable.
- Tamaño de **palabra**.



12

- Las memorias actuales están basadas en los semiconductores (transistores).
- Celda de memoria:
 - Elemento de memoria más pequeño posible.
 - Tiene dos valores estables, que pueden interpretarse como 0 o 1 (=almacena 1 bit).
 - En ella se puede escribir un dato para establecer su valor. Para ello se activa la señal Select, que selecciona la celda de memoria, y la señal de Control, que indica la operación de escritura. El dato "Data in" es escrito en la celda.
 - En ella también se puede leer el valor almacenado en la celda. Para ello se activa la señal Select, que selecciona la celda de memoria, y la señal de Control, que ahora indica la operación de lectura. El dato almacenado en la celda aparece en la salida "Sense".
- La memoria está basada en posiciones o direcciones numeradas en las que se pueden almacenar los datos.
 - La dirección se expresa en binario puro (valor positivo o cero).
 - Normalmente se asigna una dirección única a un conjunto de celdas de memoria que se denomina unidad direccionable.
 - Por defecto en esta asignatura, consideraremos que la unidad direccionable de la memoria es 1 byte. Esto quiere decir que a cada byte contenido en la memoria le corresponde una dirección que lo identifica unívocamente.
 - Ej.: En MIPS la unidad direccionable es el byte (8 bits).
- El conjunto de bits que almacenan de manera natural un dato manejable por el computador se denomina palabra.
 - El tamaño de palabra suele coincidir con el tamaño de las entradas y salidas de la ALU, de los registros de propósito general, así como del bus de datos.
 - En muchos ejercicios se indica cuántos bits tiene el bus de datos sin mencionar ni una sola vez cuál es el tamaño de palabra.
 - El tamaño de palabra suele ser una potencia de 2 del byte (1, 2, 4, 8, etc.).
 - Ej.: En MIPS el tamaño de palabra es 4 bytes (32 bits).
 - Si la unidad direccionable es la palabra, entonces son las palabras (y no los bytes sueltos) las que reciben una dirección de memoria propia.

Figura extraída del libro William Stallings, Computer Organization and Architecture, 9.^a edición, pág. 161, 2013.

ORDEN DE LOS BYTES (*ENDIANNESS*)

- Organización de los datos de varios bytes:
 - Little endian*: el byte menos significativo primero.
 - Big endian*: el byte más significativo primero.
- Ejemplo: dato **0x12345678** en la posición **0x1000** de memoria en una memoria organizada en bytes.

<i>Little endian</i>		<i>Big endian</i>	
0x1000	0x78	0x1000	0x12
0x1001	0x56	0x1001	0x34
0x1002	0x34	0x1002	0x56
0x1003	0x12	0x1003	0x78

- Se llama "*endianness*" al orden en el que se interpretan los bytes de una palabra de varios bytes.
 - En un dato que ocupa varios bytes, podemos identificar el byte más significativo (aquel al que le corresponde las potencias de 2 más altas) y el menos significativo (aquel con las potencias de 2 más bajas).
 - Cuando el byte MÁS significativo se coloca en la posición de memoria MÁS baja, se llama "*big endian*".
 - Cuando el byte MENOS significativo se coloca en la posición de memoria MÁS baja, se llama "*little endian*".
- Ejemplos:
 - Big endian*: IBM System/360, IBM System/370, IBM z/Architecture, ESA/390, PDP-10, Motorola 6800, Motorola 68000.
 - Little endian*: MOS 6502, Zilog Z80, Intel x86, Intel x86-64.
 - Algunas arquitecturas de microprocesador pueden trabajar con ambos formatos (ARM, PowerPC, DEC Alpha, PA-RISC, Arquitectura MIPS), y a veces son referidas como sistemas "*middle-endian*".
 - El simulador MARS trabaja como *little endian*, mientras que el simulador SPIM es *big endian*, siendo en ambos casos simuladores del procesador MIPS.
- La palabra "*endianness*" surge en el libro de "Los viajes de Gulliver" de Jonathan Swift (1726):
 - Gulliver se encuentra dos pueblos enfrentados entre sí por decidir por qué extremo ("*end*") debe romperse un huevo hervido: por el lado grande o por el pequeño.
 - Aplicado al orden de los bytes, este concepto fue utilizado por primera vez por el informático Danny Cohen en un famoso artículo científico de 1980 titulado "On Holy Wars and a Plea for Peace".

NOTA: Los valores expresados en base hexadecimal los indicaremos con el prefijo 0x. En algunos libros se coloca un signo \$ como prefijo, o bien, el sufijo h o el subíndice 16 con el mismo significado. Ejemplo: $0x12345678 \equiv \$12345678 \equiv 12345678h \equiv 12345678_{16}$.

ALINEACIÓN DE LOS ACCESOS A MEMORIA

○ Accesos a memoria alineados:

- La dirección de memoria es múltiplo del tamaño del dato al que se accede.
- Ej.: acceso a palabra (4 bytes) $\Rightarrow$ direcciones múltiplo de 4.
- Ej.: acceso a media palabra (2 bytes) $\Rightarrow$ direcciones múltiplo de 2.
- Los accesos a memoria son más rápidos.

○ Accesos a memoria no alineados:

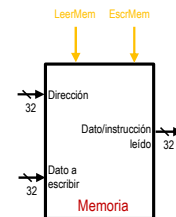
- Se hacen dos accesos a memoria y se descartan los bits sobrantes.
- Sufren tal penalización de tiempo que no suelen estar permitidos.

14

- Es habitual que la unidad mínima que puede direccionar la memoria sea 1 byte, si bien las palabras pueden estar formadas por varios bytes (2, 4, 8 bytes, etc.).
 - Cuando direccionamos un único byte, la dirección podrá tener cualquier valor (par o impar).
 - En realidad, se lee la palabra completa y después se descartan los bits sobrantes mediante un circuito especial llamado red de alineamiento.
 - Por el contrario, cuando direccionamos una palabra completa, se suelen exigir accesos a memoria alineados a palabra, lo que nos impone limitaciones en el valor de la dirección (alineamiento):
 - Palabra de 2 bytes: la memoria solo admite direcciones pares (con su último bit igual a 0).
 - Palabra de 4 bytes: la memoria solo admite direcciones múltiplo de 4 (los dos últimos bits de la dirección valen 0).
 - Palabra de 8 bytes: la memoria solo admite direcciones múltiplos de 8 (los tres últimos bits de la dirección valen 0).
 - Admitir accesos no alineados disminuye el rendimiento, porque es preciso acceder a dos palabras consecutivas para leer (o escribir) un dato de tamaño palabra no alineado, así como reordenar los bits leídos para componer el dato mediante una red de alineamiento.
- Recordemos que, en MIPS, con palabra de 4 bytes, disponemos de las instrucciones lw (carga de palabra), lh (carga de media palabra) y lb (carga de byte).
 - El valor cargado se escribe en un registro de 4 bytes.
 - Para lh y lb se realiza extensión de signo del bit 15 o del 7, respectivamente.
 - Para no realizar la extensión de signo (se rellena con 0), tenemos las instrucciones lhu y lbu, donde la "u" significa "unsigned".

ENTRADAS Y SALIDAS DE LA MEMORIA

- **Entrada/salida de datos:**
 - N bits de datos $\Rightarrow N$ bits de ancho de palabra
- **Entrada de direcciones:**
 - M bits de dirección $\Rightarrow 2^M$ unidades direccionables
- **Entradas de control:**
 - **LeerMem** (para leer la dirección)
 - **EscrMem** (para escribir en la dirección)
 - Puede haber otras, como, por ejemplo:
 - Selección del chip de memoria
 - Habilitación de la salida (buffer triestado)
 - Selección de fila/columna en chips DRAM



15

- De manera genérica, una memoria dispone de las siguientes entradas y salidas:
 - Entrada/salida de datos:
 - Es por donde sale un dato cuando es leído o por donde entra un dato que queremos escribir.
 - Normalmente esta entrada/salida está conectada al bus de datos, con lo que el ancho coincide (N bits).
 - Entrada de direcciones:
 - Es por donde entra la dirección de memoria que queremos leer o donde queremos escribir.
 - Está conectado al bus de direcciones, que tiene ancho M bits.
 - Con M bits se pueden formar 2^M direcciones diferentes (en binario puro), que son las unidades direccionables de la memoria.
 - Entradas de control:
 - Son las señales auxiliares que permiten el funcionamiento correcto de la memoria.
 - Están conectadas al bus de control.
 - En el camino de datos de MIPS estudiado en el tema anterior, tenemos las señales LeerMem y EscrMem.
 - En otras memorias más complejas puede haber otras señales como las mencionadas en la transparencia (no las vamos a estudiar).

TIPOS DE MEMORIA SEGÚN EL MODO DE ACCESO A LOS DATOS

- **Memoria de acceso aleatorio (*Random Access Memory*, RAM):**
 - El tiempo de acceso a cualquier dato es constante.
 - Memoria principal.
- **Memoria de acceso secuencial:**
 - Los datos se recorren en serie desde el principio.
 - Cinta de casete o un DAT.
- **Memoria de acceso directo:**
 - Información almacenada por zonas, que permiten un acceso más “directo” que en el caso secuencial.
 - Disco duro magnético (HDD).
- **Memoria de acceso asociativo:**
 - Se busca los datos por su valor, no por su dirección.
 - Memoria caché.

16

- Memoria de acceso aleatorio (*random access memory*, RAM):
 - Memoria con tiempo de acceso constante e independiente de la dirección física de la posición accedida.
 - Todos los ciclos duran lo mismo.
 - La memoria principal es de tipo RAM.
 - Es un exceso del lenguaje hablar de “memorias RAM” para referirnos a las memorias de semiconductores que permiten lectura y escritura para diferenciarlas de las “memorias ROM”, que solo permiten operaciones lectura.
- Memoria de acceso secuencial (*sequential access memory*):
 - Memoria con tiempo de acceso dependiente de la posición física en la que se encuentran los datos buscados.
 - Para acceder a una información es preciso recorrer toda la que hay grabada delante.
 - Es el caso de las cintas de casete o los DAT (*Digital Audio Tape*).
- Memoria de acceso directo (*direct access memory*):
 - En ella la información está almacenada por zonas y, para acceder a un dato, hay que acceder a una zona determinada para después buscar en ella el dato secuencialmente.
 - Con esta organización se puede acceder más “directamente” a los datos, comparadas con las memorias de acceso secuencial.
 - Ej.: los discos duros magnéticos (HDD) disponen de pistas (anillos concéntricos), las cuales están divididas en sectores (“quesitos” del disco).
 - La aguja lectora debe llegar primero desde su posición actual hasta la pista correcta, pasando por todas las pistas intermedias.
 - A continuación, una vez en la pista correcta, debe recorrer todos los sectores intermedios hasta llegar al que nos interesa.
- Memoria de acceso asociativo:
 - Es un tipo especial de memoria que permite buscar la información deseada por su contenido, no por su dirección.
 - Ej.: una memoria caché totalmente asociativa, en la que hay que comprobar si en alguna posición de la misma está almacenada una cadena de bits conocida llamada “etiqueta”.
 - Esta operación de búsqueda se realiza en todas las posiciones de la caché a la vez.
 - En caso de encontrarse la etiqueta en alguna posición, se denomina “acierto de caché”.
 - En caso contrario, se trata de un “fallo de caché”.

ALGUNOS TIPOS DE MEMORIAS

- Tipos de memorias de semiconductores:
 - **ROM:** Memoria de (casi) solo lectura.
 - **SRAM:** Memoria RAM estática.
 - **DRAM:** Memoria RAM dinámica.
- Actualmente también se utilizan soportes magnéticos (discos, cintas,...) y discos de estado sólido.

Tecnología	Tiempo de acceso (ns)	\$ por GB (2012)
SRAM	0,5 – 2,5	500 – 1000
DRAM	50 – 70	10 – 20
Flash	5000 – 50 000	0,75 – 1,00
Disco magnético	5 000 000 – 20 000 000	0,05 – 0,10

17

- Tipos de memoria según la volatilidad de los datos almacenados:
 - **Memoria volátil:**
 - Su contenido se borra cuando cesa la alimentación de energía.
 - Ej.: registros, memoria principal, memoria caché.
 - **Memoria no volátil:**
 - Su contenido se mantiene aun cuando cese la alimentación.
 - Ej.: disco duro magnético (HDD), unidad de estado sólido (SSD), memoria flash, CD-ROM, DVD, Blu-Ray, etc.
- Con esto en mente, podemos hacer una clasificación de las memorias:
 - **Memoria de solo lectura (Read-Only Memory, ROM):**
 - Originalmente, su contenido solo se puede leer, pero no escribir.
 - En el proceso de fabricación de la misma es cuando se graban los datos.
 - Con el tiempo, se han desarrollado memorias ROM modificables o “programables” mediante procesos físicos, como son baños de luz de determinada longitud de onda (EPROM) o mediante señales eléctricas (EEPROM).
 - Una evolución de las EEPROM son las llamadas memorias Flash, muy populares hoy en día, que se utilizan como dispositivo de almacenamiento extraíble (tanto lectura como escritura).
 - **Memoria RAM estática (SRAM):**
 - Tipo de memoria RAM basada en biestables (componentes digitales para almacenamiento de información).
 - Cuando se escribe un dato, dicho dato permanece inalterado hasta que escribimos un nuevo dato sobre él o hasta que cesa la alimentación de energía (son volátiles).
 - Las memorias SRAM son más rápidas que las DRAM, aunque tienen menos capacidad y son más caras.
 - Una celda de memoria SRAM ocupa más espacio que las DRAM porque se construye con más transistores. A igualdad de superficie en la oblea de silicio, alcanzan menos capacidad de memoria.
 - Actualmente se utilizan para implementar las memorias cachés en el chip del procesador.
 - **Memoria RAM dinámica (DRAM):**
 - Tipo de memoria RAM basada en condensadores (componentes analógicos para almacenamiento de información). A grandes rasgos, son como “cubos de agua” con un pequeño agujero (tienen pérdidas). Si el cubo está medio lleno, es un 1; si está medio vacío, es un 0.
 - Es una memoria necesariamente volátil en la que la información almacenada se va degradando a lo largo del tiempo, a no ser que su contenido se restaure mediante una operación denominada “refresco”.
 - La DRAM usa un área menor por bit de memoria y, en consecuencia, tienen más capacidad en la misma área de silicio comparada con la SRAM.
 - A cambio, es más lenta que la SRAM.
 - Actualmente se utilizan en la memoria principal.

La tabla está tomada del libro Patterson & Hennessy, Computer Organization and Design, 5.ª edición, pág. 378, 2014. Más info: sección B.9 (apéndice B).

ÍNDICE

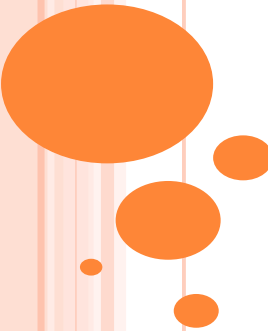
- Introducción a la jerarquía de memoria
 - Introducción
 - Tecnologías de memoria
 - **Parámetros del rendimiento**
- Memoria caché
 - Introducción
 - Elementos de diseño
 - Medición del rendimiento

PARÁMETROS DE RENDIMIENTO

- **Tamaño máximo (máxima memoria direccionable):** 2 elevado al ancho del bus de direcciones.
- **Tiempo de ciclo:** el transcurrido entre dos accesos consecutivos de memoria.
- **Tiempo de acceso (latencia):** el transcurrido desde que se proporciona la dirección y la señal de lectura hasta que sale el dato leído.
- **Ancho de banda:** número de bytes por unidad de tiempo que pueden transferirse.

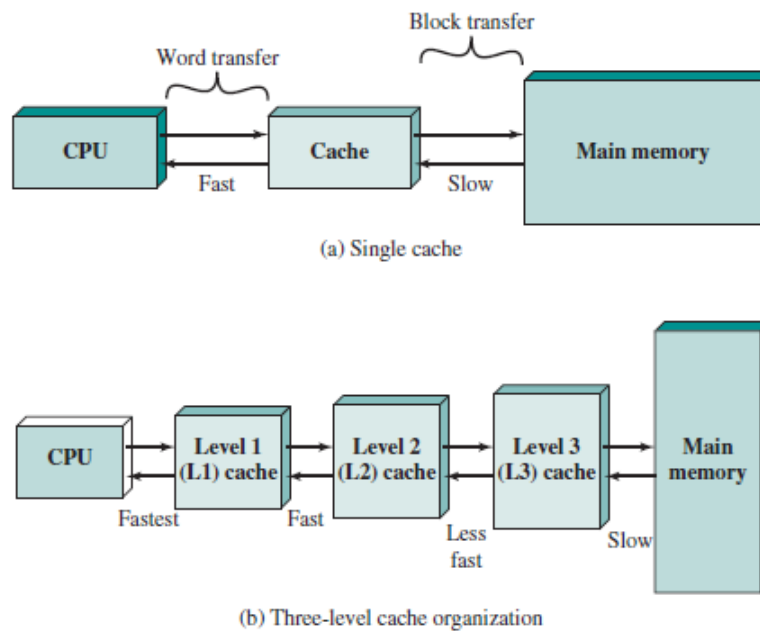
19

- Algunos parámetros de rendimiento de la memoria:
 - El tamaño máximo de la memoria o la cantidad de memoria direccionable por el procesador.
 - Corresponde al número máximo de unidades direccionables (normalmente bytes) que a las que somos capaces de acceder.
 - Depende del tamaño del bus de direcciones (M bits).
 - Hay 2^M unidades direccionables (normalmente bytes).
 - Este tamaño máximo difiere de la memoria instalada, que depende del número de tarjetas de memoria RAM conectadas a la placa base.
 - Obviamente la máxima memoria instalada es la cantidad de memoria direccionable.
 - Tiempo de ciclo:
 - Tiempo mínimo que debe transcurrir entre dos accesos consecutivos a memoria.
 - Incluye, por ejemplo, el tiempo de refresco en las memorias RAM dinámicas.
 - Tiempo de acceso (latencia):
 - Tiempo que se tarda en acceder a un dato.
 - Puede ser diferente para el caso de leer que para escribir, aunque suelen coincidir.
 - Ancho de banda (*bandwidth*) :
 - El número de datos transmitidos por unidad de tiempo.
 - Se interpreta como una velocidad de transferencia.
 - Se mide en bits/s, bytes/s o un múltiplo de ellas.
 - Depende de diversos factores:
 - Anchura de la memoria y de los buses.
 - Tiempo de acceso a la memoria.
 - Ancho de banda del bus.
 - Arquitectura interna de la memoria.
 - Etc.



MEMORIA CACHÉ

INTRODUCCIÓN



21

- La memoria caché es un nivel de la jerarquía de memoria entre la CPU y la memoria principal.
 - Se divide en bloques o líneas de caché.
 - A su vez, la memoria principal se divide en bloques del mismo tamaño.
 - La transferencia de información entre caché y memoria principal se realiza mediante bloques de datos.
 - Con esta tecnología se consigue mejorar el rendimiento gracias al principio de localidad.
- Supongamos que el bus de datos tiene n bits de ancho y que la memoria principal es direccionable a nivel de palabra.
 - La memoria principal contiene 2^n palabras.
 - Si agrupamos las palabras en bloques de tamaño K , el número total de bloques de la memoria será $M = 2^n/K$.
- Por otro lado, la memoria caché se organiza en m líneas en las que caben K palabras (hay bits adicionales para la etiqueta, el bit de validez, el bit sucio, etc., que ignoraremos de momento).
 - Por tanto, el tamaño de línea de caché coincide con el tamaño de bloque de memoria principal, en este caso, K palabras.
- Se tiene que m (n.º de líneas de caché) $\ll M$ (n.º de bloques de memoria principal).
- La transferencia entre dos niveles de la jerarquía de memoria se realiza siempre a nivel de bloque, no de palabra individual.

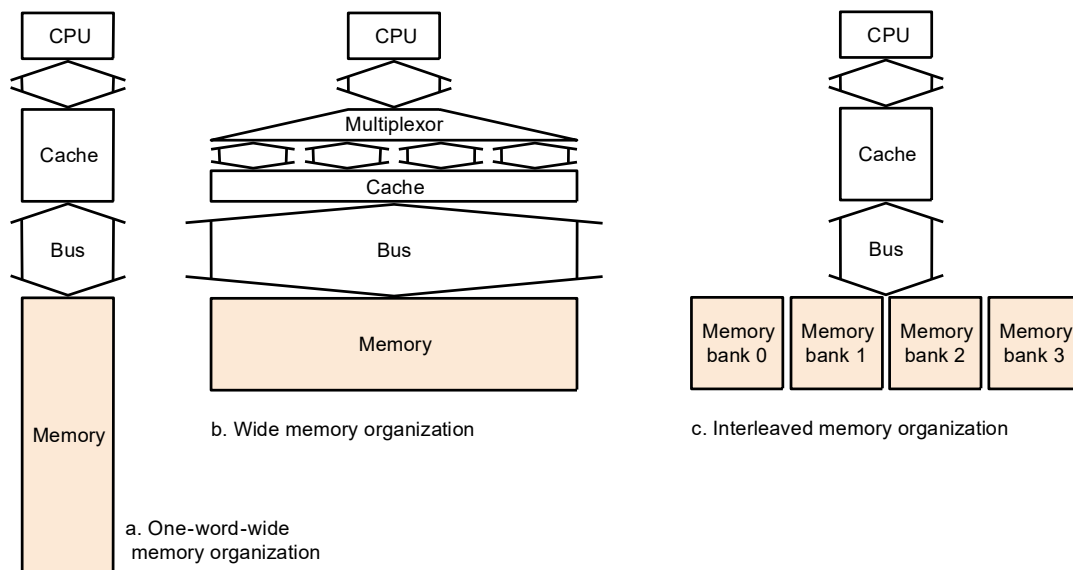
Figura extraída del libro William Stallings, Computer Organization and Architecture, 9.ª edición, pág. 120, 2013.

TIPOS DE FALLOS DE CACHE: LAS TRES C

- **Fallo obligatorio (*Compulsory miss*)**
- **Fallo por capacidad (*Capacity miss*)**
- **Fallo por conflicto (*Conflict miss*)**

- Los tres tipos de fallos de una caché:
 - **Fallo obligatorio (*Compulsory miss*):**
 - Se produce cuando el programa hace su primer acceso a memoria al comenzar la ejecución, con lo que la caché está vacía y se produce el fallo.
 - Es inevitable.
 - **Fallo por capacidad (*Capacity miss*):**
 - Debido a la capacidad limitada de la caché, un bloque puede ser descartado y reemplazado, aunque vayamos a acceder a él próximamente.
 - **Fallo por conflicto (*Conflict miss*):**
 - Ocurre cuando la misma línea de caché corresponde a varios bloques de memoria principal, de modo que tienen que ser reemplazados continuamente.

MEMORIA PRINCIPAL FRENTE A MEMORIA CACHE



23

- Caso a:
 - La memoria principal tiene un ancho de palabra que coincide con el bus de datos.
 - La caché también se comunica con el procesador con el mismo ancho de datos.
- Caso b:
 - Disponemos de una memoria de ancho ampliado, que permite transmitir varias palabras a la vez a través del bus.
 - Sin embargo, la caché se sigue comunicando con el procesador con el tamaño de palabra.
- Caso c:
 - La memoria está organizada en bancos (chips separados que pueden leerse en paralelo).
 - En este ejemplo, podemos leer 4 palabras con la latencia de 1 sola.
 - El ancho del bus de datos es solo tamaño palabra, lo cual obliga a transmitir las palabras de una en una.

Figura extraída del libro Patterson & Hennessy, Computer Organization and Design, 3.^a edición, pág. 489, 2005.

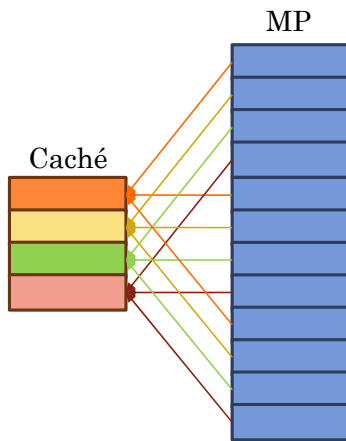
ÍNDICE

- Introducción a la jerarquía de memoria
 - Introducción
 - Tecnologías de memoria
 - Parámetros del rendimiento
- Memoria caché
 - Introducción
 - **Elementos de diseño**
 - Medición del rendimiento

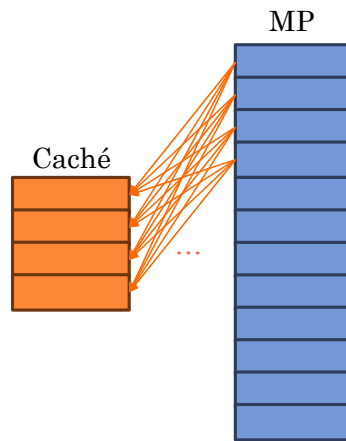
ELEMENTOS DE DISEÑO DE LA CACHE

- **Función de correspondencia**
- Algoritmo de reemplazo
- Política de lectura/escritura
- Tamaño de línea
- Tamaño de caché
- Otros: niveles, uso compartido, ubicación

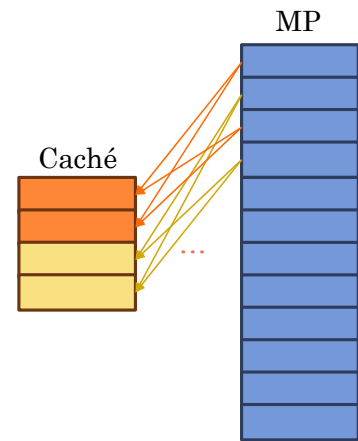
FUNCIÓN DE CORRESPONDENCIA



Correspondencia directa



Totalmente asociativa



Asociativa por
conjuntos

26

- Hay muchas menos líneas de caché que bloques de memoria principal (MP).
- La función de correspondencia permite decidir en qué línea de la caché podemos ubicar un bloque de memoria.
- Hay varias estrategias:
 - Correspondencia directa.
 - Un bloque de MP se asigna de manera unívoca a una única línea de caché.
 - Dicho de otro modo, a una línea de caché pueden asignarse diferentes bloques de MP, que competirán entre sí por hacerse con la línea.
 - Es el sistema más rígido, pero el más fácil de implementar.
 - Para comprobar un acierto de caché, solo hay que mirar en una línea.
 - Totalmente asociativa.
 - Un bloque de MP puede asignarse a cualquier línea de caché.
 - Es el sistema más flexible, pero el más difícil de implementar.
 - Para comprobar un acierto de caché, hay que mirar a la vez en todas las líneas. Esto se hace en paralelo con un comparador asociado a cada entrada de la caché.
 - En la práctica, se elige la línea que diga el algoritmo de reemplazo (ver más adelante).
 - Asociativa por conjuntos.
 - La caché se organiza en conjuntos, que a su vez tienen varias vías (todos los conjuntos tienen el mismo número de vías).
 - Un bloque de MP se asigna de manera unívoca a un conjunto.
 - Dentro del conjunto, dicho bloque puede asignarse a cualquier vía.
 - En la práctica, se elige la vía que diga el algoritmo de reemplazo (ver más adelante), pero el conjunto es unívoco.
 - Es un sistema híbrido, de compromiso, que al final es el más usado en la práctica.
 - Para comprobar un acierto de caché, solo hay que mirar en las vías de un conjunto.
 - En la imagen la caché que se muestra tiene 2 conjuntos (naranja y amarillo) de dos vías cada uno.

CORRESPONDENCIA DIRECTA

$$\text{N.º de línea} = (\text{N.º de bloque}) \bmod (\text{N.º total de líneas de caché})$$

- Unidad direccionable de memoria principal: 1 byte.
- Tamaño de las direcciones de memoria: n bits.
- Tamaño de palabra: 2^p unidades direccionables (bytes).
- Tamaño de bloque = tamaño de línea = 2^q palabras.
- Número de líneas de caché = 2^r .

Dirección de bloque		Desplazamiento de bloque	
Etiqueta ($n-r-q-p$ bits)	Índice de línea (r bits)	Desplazamiento de palabra (q bits)	Desplazamiento de byte (p bits)
Etiqueta identificadora	Identifican la línea de caché	Identifican la palabra en el bloque	Identifican el byte en la palabra

27

- No hay elección posible para la línea de caché, ya que se calcula con la fórmula de la transparencia.
 - En la práctica, el número de línea se encuentra codificado en uno de los campos en los que puede dividirse la dirección de memoria en binario (entre la etiqueta y el desplazamiento de palabra).
- Aprovechando que todos los tamaños típicos del computador se eligen para que sean potencia perfecta de 2, el cálculo de la operación módulo es muy sencillo y se consigue simplemente viendo una trama de bits de la dirección de memoria.
- Para direcciones de memoria de n bits, con unidad direccionable a nivel de byte (lo habitual), se cumple:
 - El tamaño de palabra es 2^p bytes. Luego se usan los p bits menos significativos de la dirección para identificar el byte dentro de la palabra.
 - El tamaño de bloque es 2^q palabras. Luego se usan los siguientes q bits de la dirección para identificar la palabra dentro del bloque.
 - El número de líneas de la caché es 2^r . Luego los siguientes r bits de la dirección especifican a qué línea de caché se asigna este bloque de memoria.
 - Los $(n-r-q-p)$ bits restantes, los más significativos, se utilizan como etiqueta para identificar el bloque en la caché.
- Nótese que, si el tamaño de bloque es de solo 1 palabra, entonces el valor de q es 0, ya que $2^0 = 1$.
 - Eso significaría que no se utilizaría ningún bit de la dirección para indicar el desplazamiento de palabra dentro del bloque.
- Si, por el contrario, la unidad direccionable fuera la palabra, entonces p valdría 0 y no podríamos hacer referencia a un byte concreto de la palabra, solo a la palabra completa.

CORRESPONDENCIA DIRECTA: EJEMPLO

- Caché de 256 líneas, bloques de 16 palabras y palabras de 4 bytes, con direcciones de memoria de 32 bits.
 - $r = 8$, ya que $2^8 = 256$.
 - $q = 4$, ya que $2^4 = 16$.
 - $p = 2$, ya que $2^2 = 4$.
 - Tamaño de las etiquetas = $32 - 8 - 4 - 2 = 18$ bits.

- Sea la dirección 0xABCDEF01, es decir:

1010 1011 1100 1101 11|10 1111 00|00 00|01

- **Etiqueta:** 10 1010 1111 0011 0111 = 0x2AF37
- **Línea de caché:** 1011 1100 = 0xBC = 188 (no hay elección posible con la línea)
- **Desplazamiento de palabra:** 0000 = 0
- **Desplazamiento de byte:** 01 = 1

28

Supongamos que la dirección de memoria a la que se quiere acceder vale 2 882 400 001 en decimal o 0xABCDEF01 en hexadecimal. Este valor corresponde en binario a 1010 1011 1100 1101 1110 1111 0000 0001. Esta dirección se descompone en los siguientes campos:

- Etiqueta: 10 1010 1111 0011 0111 = 0x2AF37
- Línea de caché: 1011 1100 = 0xBC = 188
- Desplazamiento de palabra: 0000 = 0
- Desplazamiento de byte: 01 = 1

Esto significa que queremos acceder al byte 1 (de 4 posibles) de la palabra 0 (de 16 posibles) del bloque. El bloque que contiene este dato se va a asignar a la línea de caché 188 de las 256 posibles y para distinguirla de otros bloques que van obligatoriamente a la misma línea, se almacena también la etiqueta 0x2AF37.

Por la fórmula del módulo, supongamos que nuestra memoria principal tiene 4 GB (ya que las direcciones son de 32 bits) y que se divide en bloques de 64 bytes (16 palabras de 4 bytes cada una). Esto significa que nuestra memoria está dividida en 67 108 864 bloques. El bloque 0 abarca las direcciones de 0 a 63; el bloque 1, de 64 a 127, etc. Si dividimos la dirección de memoria 2 882 400 001 entre 64, vemos que sale 45 037 500,015625. Esto significa que se trata de una palabra que se encuentra en el bloque 45 037 500. En este caso, el número de línea de caché al que se asigna este bloque de memoria principal se calcula como $45\,037\,500 \bmod 256 = 188$, que es lo mismo que nos ha salido por el otro método.

TOTALMENTE ASOCIATIVA

N.º de línea = Cualquiera

- Unidad direccionable de memoria principal: 1 byte.
- Tamaño de las direcciones de memoria: n bits.
- Tamaño de palabra: 2^p unidades direccionables (bytes).
- Tamaño de bloque = tamaño de línea = 2^q palabras.
- Número de líneas de caché = 2^r .

Dirección de bloque	Desplazamiento de bloque	
Etiqueta ($n-q-p$ bits)	Desplazamiento de palabra (q bits)	Desplazamiento de byte (p bits)

29

- Para direcciones de memoria de n bits, con unidad direccionable a nivel de byte, se cumple:
 - El tamaño de palabra es 2^p bytes. Luego se usan los p bits menos significativos de la dirección para especificar la posición del byte dentro de la palabra.
 - El tamaño de bloque es 2^q palabras. Luego se usan los siguientes q bits menos significativos de la dirección para especificar la posición de la palabra dentro del bloque.
 - El resto de bits ($n-p-q$), los más significativos de la dirección, se utilizan como etiqueta para identificar el bloque en la caché.

TOTALMENTE ASOCIATIVA: EJEMPLO

- Caché de 256 líneas, bloques de 16 palabras y palabras de 4 bytes, con direcciones de memoria de 32 bits.

- $q = 4$, ya que $2^4 = 16$.
- $p = 2$, ya que $2^2 = 4$.
- Tamaño de las etiquetas = $32 - 4 - 2 = 26$ bits.

- Sea la dirección 0xABCDEF01, es decir:

1010 1011 1100 1101 1110 1111 0000 0001

- **Etiqueta:** 10 1010 1111 0011 0111 1011 1100 = 0x2AF37BC
- **Desplazamiento de palabra:** 0000 = 0
- **Desplazamiento de byte:** 01 = 1
- El bloque correspondiente a esta dirección de memoria se podrá asignar a cualquier línea de caché (la que diga el algoritmo de reemplazo).

30

Supongamos que la dirección de memoria a la que se quiere acceder vale 2 882 400 001 en decimal o 0xABCDEF01 en hexadecimal. Este valor corresponde en binario a 1010 1011 1100 1101 1110 1111 0000 0001. Esta dirección se descompone en los siguientes campos:

- Etiqueta: 10 1010 1111 0011 0111 1011 1100 = 0x2AF37BC
- Desplazamiento de palabra: 0000 = 0
- Desplazamiento de byte: 01 = 1

Esto significa que queremos acceder al byte 1 (de 4 posibles) de la palabra 0 (de 16 posibles) del bloque. El bloque que contiene este dato se va a asignar a cualquier línea de caché de las 256 posibles y para distinguirla de otros bloques, se almacena también la etiqueta 0x2AF37BC.

ASOCIATIVA POR CONJUNTOS

$N.^\circ \text{ de conjunto} = (N.^\circ \text{ de bloque}) \text{ módulo } (N.^\circ \text{ total de conjuntos en la caché})$

- Unidad direccionable de memoria principal: 1 byte.
- Tamaño de las direcciones de memoria: n bits.
- Tamaño de palabra: 2^p unidades direccionables (bytes).
- Tamaño de bloque = tamaño de línea = 2^q palabras.
- Número de conjuntos de caché = 2^s .
- Número de vías por conjunto: v .

Dirección de bloque		Desplazamiento de bloque	
Etiqueta ($n-s-q-p$ bits)	Índice de conjunto (s bits)	Desplazamiento de palabra (q bits)	Desplazamiento de byte (p bits)
Sirve para verificar si el bloque buscado está en la caché	Identifican el conjunto	Identifican la palabra en el bloque	Identifican el byte en la palabra

31

- El conjunto se calcula como si fuera una caché de correspondencia directa (sin elección posible).
 - En la práctica, el número de conjunto se encuentra codificado en uno de los campos en los que puede dividirse la dirección de memoria en binario (entre la etiqueta y el desplazamiento de palabra).
- Dentro de las vías de dicho conjunto, se elige como si fuera de correspondencia totalmente asociativa (la que diga el algoritmo de reemplazo).
- Para direcciones de memoria de n bits, con unidad direccionable a nivel de byte, se cumple:
 - El tamaño de palabra es 2^p bytes. Por tanto, se usan los p bits menos significativos de la dirección para especificar el byte dentro de la palabra.
 - El tamaño de bloque es 2^q palabras. Por tanto, se usan los siguientes q bits menos significativos de la dirección para especificar la palabra dentro del bloque.
 - El número de conjuntos de la caché es 2^s . Luego se usan los siguientes s bits de la dirección para indicar a qué conjunto de caché se asigna el bloque.
 - El resto de bits ($n-p-q-s$), los más significativos de la dirección, sirven como etiqueta para identificar el bloque en la caché.
- ¡OJO! En la descomposición de la dirección de memoria en campos NO interviene el número de vías que hay en cada conjunto (v).
- Nótese que:
 - Una caché con correspondencia directa se puede ver como 1-asociativa, es decir, se compone de conjuntos de 1 sola vía.
 - Una caché totalmente asociativa puede entenderse como si tuviera 1 solo conjunto con tantas vías como líneas.
 - La distinción de los tipos de caché en tres clases es, por tanto, artificial, ya que todas las cachés pueden considerarse asociativas por conjuntos.

ASOCIATIVA POR CONJUNTOS: EJEMPLO

- Caché de 256 líneas, 64 conjuntos, 4 vías por conjunto, bloques de 16 palabras y palabras de 4 bytes, con direcciones de memoria de 32 bits.

- $s = 6$, ya que $2^6 = 64$.
- $q = 4$, ya que $2^4 = 16$.
- $p = 2$, ya que $2^2 = 4$.
- Tamaño de las etiquetas = $32 - 6 - 4 - 2 = 20$ bits.

- Sea la dirección 0xABCDEF01, es decir:

1010 1011 1100 1101 1110 | 1111 00 | 00 00 | 01

- **Etiqueta:** 1010 1011 1100 1101 1110 = 0xABCDE
- **Conjunto:** 11 1100 = 0x3C = 60
- **Desplazamiento de palabra:** 0000 = 0
- **Desplazamiento de byte:** 01 = 1
- El bloque correspondiente a esta dirección de memoria se podrá asignar a cualquiera de las 4 vías del conjunto 60 (la que diga el algoritmo de reemplazo).

32

Supongamos que la dirección de memoria a la que se quiere acceder vale 2 882 400 001 en decimal o 0xABCDEF01 en hexadecimal. Este valor corresponde en binario a 1010 1011 1100 1101 1110 1111 0000 0001. Esta dirección se descompone en los siguientes campos:

- Etiqueta: 1010 1011 1100 1101 1110 = 0xABCDE
- Conjunto: 11 1100 = 0x3C = 60
- Desplazamiento de palabra: 0000 = 0
- Desplazamiento de byte: 01 = 1

Esto significa que queremos acceder al byte 1 (de 4 posibles) de la palabra 0 (de 16 posibles) del bloque. El bloque que contiene este dato se va a asignar a cualquiera de las 4 vías del conjunto 60 de los 64 posibles y para distinguirla de otros bloques, se almacena también la etiqueta 0xABCDE.

Por la fórmula del módulo, supongamos que nuestra memoria principal tiene 4 GB (ya que las direcciones son de 32 bits) y que se divide en bloques de 64 bytes (16 palabras de 4 bytes cada una). Esto significa que nuestra memoria está dividida en 67 108 864 bloques. El bloque 0 abarca las direcciones de 0 a 63; el bloque 1, de 64 a 127, etc. Si dividimos la dirección de memoria 2 882 400 001 entre 64, vemos que sale 45 037 500,015625. Esto significa que se trata de una palabra que se encuentra en el bloque 45 037 500. En este caso, el número de conjunto de caché al que se asigna este bloque de memoria principal se calcula como $45\,037\,500 \text{ módulo } 64 = 60$, que es lo mismo que nos ha salido por el otro método.

ELEMENTOS DE DISEÑO DE LA CACHE

- Función de correspondencia
- **Algoritmo de reemplazo**
- Política de lectura/escritura
- Tamaño de línea
- Tamaño de caché
- Otros: niveles, uso compartido, ubicación

ALGORITMO DE REEMPLAZO

- Aleatorio
- El utilizado menos recientemente (*Least-Recently Used*, LRU)
- Cola (*First-In-First-Out*, FIFO)
- El utilizado menos frecuentemente (*Least Frequently Used*, LFU)
- Óptimo

34

- Algoritmos de reemplazo:
 - Aleatorio:
 - Se elige una posición al azar de entre todas las posibles, sin tener en cuenta ningún grado de utilización.
 - Es una técnica sencilla y casi tan buena como las basadas en el uso.
 - El utilizado menos recientemente (*Least-Recently Used*, LRU):
 - Es quizá la técnica más efectiva.
 - Se sustituye el bloque de caché que lleva más tiempo sin ser referenciado.
 - Es fácil de implementar en una caché asociativa por conjuntos de dos vías.
 - Para ello tendríamos un bit “uso”.
 - Al referenciar una línea de caché en una de las vías del conjunto, su bit “uso” se pone a 1, mientras que el de la otra vía se pone a 0.
 - Cuando se transfiere un dato de memoria a la caché, se sustituye la línea con bit “uso” igual a 0, ya que esta lleva más tiempo sin usarse.
 - Gracias a la localidad temporal, se espera que los datos más recientes se reutilicen con más probabilidad que los más lejanos en el tiempo.
 - El problema es que esta técnica resulta cara de implementar cuando la asociatividad es alta.
 - Cola (*First-In-First-Out*, FIFO):
 - El primero en entrar es el primero en salir (cola), es decir, se sustituye aquel bloque del conjunto que lleva más tiempo en la caché (independientemente de cuándo fue referenciado por última vez).
 - El utilizado menos frecuentemente (*Least Frequently Used*, LFU):
 - En este caso se lleva la cuenta de cuántas veces se referencia cada uno de los bloques.
 - Se sustituye el bloque que se ha referenciado menos veces.
 - Óptimo (ideal):
 - Se reemplaza el bloque que no se va a utilizar durante más tiempo.
 - El problema es que habría que tener una previsión de referencias futuras, lo cual no suele ser posible o fácil de implementar.

ALGORITMO DE REEMPLAZO

- Fallos en caché de datos según el tamaño de la misma, la política de reemplazo y el grado de asociatividad:

Size	Associativity								
	Two-way			Four-way			Eight-way		
	LRU	Random	FIFO	LRU	Random	FIFO	LRU	Random	FIFO
16 KB	114.1	117.3	115.5	111.7	115.1	113.3	109.0	111.8	110.4
64 KB	103.4	104.3	103.9	102.4	102.3	103.1	99.7	100.5	100.3
256 KB	92.2	92.1	92.5	92.1	92.1	92.5	92.1	92.1	92.5

35

- Esta tabla muestra los resultados de un experimento con el número de fallos en la caché de datos por cada 1000 instrucciones, según el algoritmo de reemplazo (LRU, aleatorio y FIFO), para tres tamaños de caché y tres niveles de asociatividad.
 - Se ve que existe poca diferencia entre los algoritmos LRU y aleatorio para las cachés de mayor tamaño y de alta asociatividad. En rojo están marcados los mejores resultados para cada caso.
 - Estos datos corresponden a una arquitectura Alpha con tamaño de bloque de 64 bytes usando 10 *benchmarks* SPEC2000. Cinco de las pruebas son de SPECint2000 (gap, gcc, gzip, mcf, perl) y las otras cinco de SPECfp2000 (applu, art, earthquake, lucas, swim).

Tabla extraída del libro Hennessy & Patterson, Computer Architecture, A Quantitative Approach, 5.ª edición, pág. B-10, 2012.

ELEMENTOS DE DISEÑO DE LA CACHE

- Función de correspondencia
- Algoritmo de reemplazo
- **Política de lectura/escritura**
- Tamaño de línea
- Tamaño de caché
- Otros: niveles, uso compartido, ubicación

POLÍTICAS DE LECTURA

○ Fallo de caché al leer instrucciones:

- Tras el primer ciclo de la ejecución de la instrucción, el contador del programa ya vale $PC + 4$. La dirección de la instrucción que genera el fallo es igual al valor de este registro MENOS 4. La ALU se encarga de realizar la resta.
- La dirección así calculada se solicita a la memoria principal y se espera a que se complete la lectura.
- La instrucción (y todo su bloque) se traen de memoria principal a la línea de caché que corresponda, se activa el bit de validez de la línea y se escribe la etiqueta.
- Una vez tengamos la instrucción que generó el fallo, se reinicia su ejecución desde el primer ciclo.

○ Fallo de caché al leer datos:

- El proceso es similar.

37

- Modificar el control de un procesador para tener en cuenta los aciertos de caché es trivial; pero con los fallos de caché se necesita un trabajo extra.
- La propuesta básica es:
 - Parar la CPU, congelando los valores de todos los registros.
 - Un controlador separado se encarga del fallo de caché, buscando los datos en la memoria.
 - Una vez que los datos están presentes, la ejecución se reanuda en el ciclo que ha causado el fallo de caché.
 - El tratamiento del fallo de caché se hace con la unidad de control del procesador y con un controlador separado que realiza el acceso a memoria y rellena la caché.
- Si un acceso a una instrucción resulta en un fallo, el contenido del registro de instrucción no es válido.
 - Para colocar la instrucción correcta en la caché se tiene que acceder al nivel más bajo de la jerarquía de memoria para realizar una lectura.
 - Como el contador de programa se incrementa en el primer ciclo de la instrucción tanto en procesadores multiciclo (estudiados en el tema anterior) como en segmentados (no los hemos estudiado), la dirección de la instrucción que genera el fallo es igual al valor del contador de programa menos cuatro.
 - Esta resta se puede calcular utilizando la ALU, aunque se necesitará almacenamiento temporal adicional en la versión segmentada.
 - Una vez que se tiene la dirección, se necesita acceder a la memoria principal para hacer una lectura.
 - El procesador esperará la respuesta de la memoria (ya que el acceso tardará algunos ciclos), y después se escribirá la palabra, junto con el resto del bloque, en la caché.
- Una propuesta similar se puede utilizar para tratar fallos de lectura de datos.

POLÍTICAS DE ESCRITURA: ACIERTO DE CACHÉ

- Ante un **acierto** de caché en una **escritura**, hay que decidir cuándo se actualiza el dato en memoria principal:
 - **Write-through (escritura inmediata):** Se escribe el bloque tanto en la caché como en la memoria principal.
 - Se usa un *buffer* de escritura.
 - **Write-back (escritura retardada):** Se escribe solo en la caché. La escritura en la memoria se retrasa hasta el momento de sustituir el bloque en la caché.
 - **Bit sucio (*dirty bit*, D):** indica si el bloque de caché ha sido modificado (un bit sucio por cada bloque).

38

- **Write-through (escritura inmediata):**
 - Tiene un pobre rendimiento: estas escrituras pueden suponer cientos de ciclos de reloj, ralentizando al procesador.
 - Para mejorar el rendimiento se usa un *buffer* de escrituras.
 - Tras escribir los datos en la caché y en el *buffer* de escrituras, el procesador puede continuar su ejecución.
 - Las entradas en el *buffer* se liberan cuando se han escrito los datos en memoria principal.
 - Si el *buffer* está lleno, el procesador deberá parar.
 - Se puede reducir el número de paradas aumentando el tamaño del *buffer*.
- **Write-back (escritura retardada):**
 - Ofrece un mejor rendimiento que *write-through*.
 - Está indicada cuando velocidad de escritura en memoria es menor que la velocidad de generación de escrituras en procesador.
 - Sin embargo, su implementación es más compleja que la de *write-through*.
 - El bit de suciedad D solo se utiliza para *write-back*, no para *write-through*.

POLÍTICAS DE ESCRITURA: FALLO DE CACHÉ

- Ante un **fallo** de caché en una **escritura** hay que decidir qué hacer con el bloque que contiene la palabra que ha provocado el fallo:
 - **Write allocate (fetch on write):** se trae el bloque entero de la memoria principal y luego se modifica la palabra en la caché como un acierto. Suele combinarse con *write-back*.
 - **No-write allocate (write around):** se modifica el bloque en memoria principal pero no se carga en la caché. Suele combinarse con *write-through*.

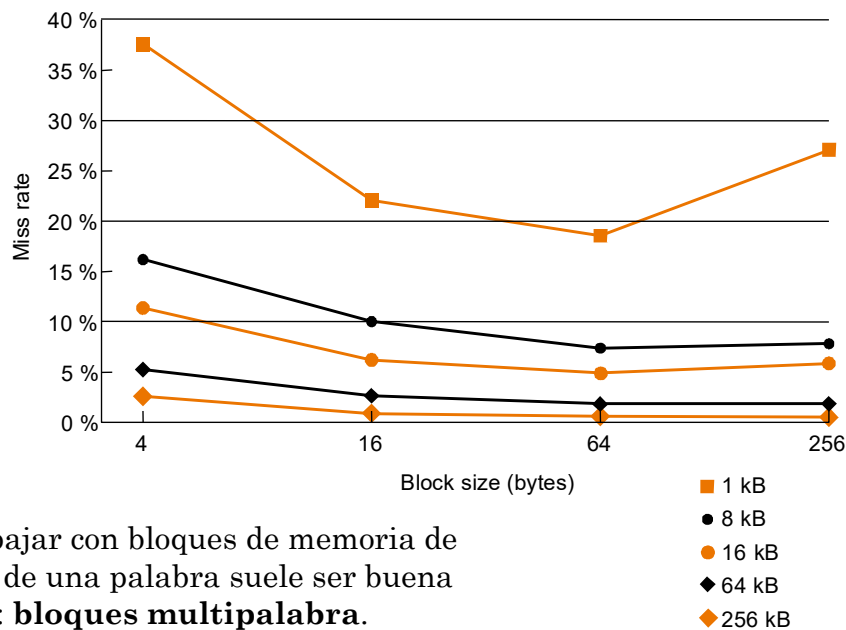
39

- Políticas de escritura de caché:
 - La distinción *write-through/write-back* se refiere a los casos de escritura con ACIERTO de caché, es decir, la palabra que vamos a escribir ya está presente en la caché.
 - En el caso *write-through*, se escribe tanto en la caché como en la memoria principal, mientras que en el caso *write-back*, se escribe solo en la caché y, con ayuda del bit de suciedad D, se modifica la memoria principal solo cuando toca sustituir ese bloque en la caché por uno nuevo.
 - La distinción *write allocate/no-write allocate* se refiere a los casos de escritura con FALLO de caché, es decir, la palabra que queremos modificar NO está en la caché.
 - En el caso *write allocate*, nos traemos en primer lugar de memoria principal el bloque que la contiene y lo guardamos en la caché. A continuación, se accede de nuevo a la caché (con un acierto) y se modifica la palabra en la caché.
 - En el caso *no-write allocate*, la palabra la modificamos directamente en la memoria principal y NO traemos su bloque a la caché.
- Ambos casos pueden combinarse. Las combinaciones más habituales son:
 - *Write-back/write allocate*: En caso de acierto, la escritura se hace en la caché (se activa el bit de suciedad) y solo se modifica la memoria principal en caso de sustituir el bloque en la caché. En caso de fallo, traemos el bloque entero desde la memoria principal para inmediatamente después modificar la palabra correspondiente en la caché (lo cual activa el bit de suciedad). Accesos posteriores a palabras del mismo bloque supondrán un acierto, ya que el bloque más reciente está en la caché, con lo que ahorraremos accesos a memoria principal. Como puede verse, tanto en caso de acierto como de fallo en las escrituras se activa el bit D.
 - *Write-through/no-write allocate*: En caso de acierto, se modifica tanto la caché como la memoria principal. En caso de fallo, se modifica directamente la memoria principal y no se trae el bloque a la caché. Accesos posteriores a palabras del mismo bloque se modifican directamente en la memoria principal (al fin y al cabo, usamos *write-through* y vamos a tener que escribir el dato en la memoria principal), con lo que se ahorra algo de tiempo al no tener que traernos el bloque a la caché.
- Las otras combinaciones no tienen mucho sentido o son ineficientes:
 - *Write-back/no-write allocate*: En caso de acierto, la escritura se hace en la caché (se activa el bit de suciedad) y solo se modifica la memoria principal en caso de sustituir el bloque en la caché. En caso de fallo, modificamos directamente la memoria principal y no se trae el bloque a la caché. Para accesos posteriores a palabras del mismo bloque, se volverá a producir un fallo de caché, luego no hemos mejorado nada, con lo que es una combinación muy ineficiente.
 - *Write-through/write allocate*: En caso de acierto, se modifica tanto la caché como la memoria principal. En caso de fallo, traemos el bloque entero desde la memoria principal para inmediatamente después modificar la palabra correspondiente en la caché, pero también en la memoria principal. Para accesos posteriores a palabras del mismo bloque, se producirá un acierto de caché, pero con esta combinación debemos seguir escribiendo tanto en la caché como en la memoria principal, lo cual no nos proporciona ningún ahorro.

ELEMENTOS DE DISEÑO DE LA CACHE

- Función de correspondencia
- Algoritmo de reemplazo
- Política de lectura/escritura
- **Tamaño de línea**
- Tamaño de caché
- Otros: niveles, uso compartido, ubicación

TAMAÑO DE LÍNEA



Trabajar con bloques de memoria de más de una palabra suele ser buena idea: **bloques multipalabra**.

41

- Gráfica con resultados experimentales de calcular la tasa de fallo en función del tamaño de las líneas (o bloques), para distintos tamaños de caché.
 - Para una cache de 1 kB, los bloques de 256 bytes generan más fallos de caché que tamaños menores.
 - Los bloques grandes favorecen la localidad espacial, pero no podemos pasarnos.
 - Con bloques muy grandes puede ocurrir que, al haber mucha competencia por ellos, sean expulsados antes de que se referencien muchas de las palabras que contienen.
 - Por otro lado, al aumentar el tamaño de bloque también pierde sentido la localidad espacial (palabras muy separadas pueden no estar relacionadas entre sí).
 - En general, la tasa de fallos desciende cuando se incrementa el tamaño de bloque.
 - Al llevar un bloque a la caché se llevan palabras situadas en memoria de forma consecutiva.
 - Cumpliendo el principio de localidad espacial, dichas palabras no provocarán fallos de caché.
 - Sin embargo, si los bloques son muy grandes, aumenta la tasa de fallos, ya que habrá muy pocos bloques en la caché, lo que producirá una fuerte competencia.
 - Otro problema de aumentar el tamaño de bloque es que crece el coste de un fallo.
 - Recordemos que el tiempo de penalización por fallo es el tiempo necesario para buscar el bloque en la memoria principal y llevarlo a la caché.
 - Este tiempo se descompone en la latencia de la primera palabra (el tiempo que tarda la memoria en proporcionarnos la primera palabra del bloque, que es un valor fijo) y el tiempo de transferencia para el resto del bloque, el cual será mayor cuanto mayor sea el tamaño de bloque.

ELEMENTOS DE DISEÑO DE LA CACHE

- Función de correspondencia
- Algoritmo de reemplazo
- Política de escritura
- Tamaño de línea
- **Tamaño de caché**
- Otros: niveles, uso compartido, ubicación

TAMAÑO DE CACHÉ

- El tamaño total de caché es el producto entre:
 - Número de líneas de caché.
 - Si es asociativa por conjuntos: número de conjuntos $\times$ número de vías por conjunto (asociatividad).
 - Tamaño de la línea de caché.
 - Tamaño de etiqueta (depende de la función de correspondencia).
 - Tamaño de bloque de datos de memoria (2^q palabras $\times$ 2^p bytes/palabra).
 - Bits adicionales:
 - Bit de validez (V): siempre está.
 - Bit sucio (D): solo si usamos la política *write-back*.
 - Bits de uso reciente (LRU): solo si es totalmente asociativa o por conjuntos.



43

- Memoria principal:
 - Capacidad grande.
 - Tiempo de acceso grande.
 - Menor coste/bit.
- Memoria caché:
 - Capacidad pequeña.
 - Tiempo de acceso bajo
 - Mayor coste/bit.
- Queremos una caché lo suficientemente grande para que el tiempo de acceso sea parecido al de la caché sola, pero también lo suficientemente pequeña para que el coste/bit total se acerque al de la memoria principal.

ELEMENTOS DE DISEÑO DE LA CACHE

- Función de correspondencia
- Algoritmo de reemplazo
- Política de lectura/escritura
- Tamaño de línea
- Tamaño de caché
- **Otros: niveles, uso compartido, ubicación**

OTROS ASPECTOS

- **Niveles de la caché en la jerarquía:**
 - Uno
 - Varios
- **Cachés para datos e instrucciones:**
 - Unificada
 - Separada
- **Cachés para los núcleos del procesador:**
 - Compartido
 - De uso exclusivo
- **Ubicación física de la caché:**
 - *On-chip*
 - *Off-chip*

45

- Cachés multinivel:
 - La memoria caché se divide en varios niveles siguiendo la misma política que en el resto de la jerarquía de memoria: cuando más cerca del procesador, más rápido y más pequeño.
 - Hoy en día suele haber 3 o 4 niveles de caché.
 - Se accede primero al nivel superior de caché y solo se desciende al nivel inmediatamente inferior en caso de fallo de caché.
- Cachés para datos e instrucciones:
 - Compartida (única o unificada) para datos e instrucciones.
 - Separada (dedicada o partida) para datos o instrucciones.
 - La tendencia actual es hacia cachés separadas.
- Según los *cores* o núcleos del procesador:
 - Compartida entre los *cores*.
 - De uso exclusivo para cada *core*.
- Según dónde se ubiquen físicamente las cachés:
 - Caché *on-chip*: En el mismo chip que el procesador.
 - Caché *off-chip*: En distinto chip al procesador.

Processor	Type	Year of Introduction	L1 Cache ^a	L2 Cache	L3 Cache
IBM 360/85	Mainframe	1968	16–32 kB	—	—
PDP-11/70	Minicomputer	1975	1 kB	—	—
VAX 11/780	Minicomputer	1978	16 kB	—	—
IBM 3033	Mainframe	1978	64 kB	—	—
IBM 3090	Mainframe	1985	128–256 kB	—	—
Intel 80486	PC	1989	8 kB	—	—
Pentium	PC	1993	8 kB/8 kB	256–512 kB	—
PowerPC 601	PC	1993	32 kB	—	—
PowerPC 620	PC	1996	32 kB/32 kB	—	—
PowerPC G4	PC/server	1999	32 kB/32 kB	256 kB to 1 MB	2 MB
IBM S/390 G6	Mainframe	1999	256 kB	8 MB	—
Pentium 4	PC/server	2000	8 kB/8 kB	256 kB	—
IBM SP	High-end server/ supercomputer	2000	64 kB/32 kB	8 MB	—
CRAY MTA ^b	Supercomputer	2000	8 kB	2 MB	—
Itanium	PC/server	2001	16 kB/16 kB	96 kB	4 MB
Itanium 2	PC/server	2002	32 kB	256 kB	6 MB
IBM POWER5	High-end server	2003	64 kB	1.9 MB	36 MB
CRAY XD-1	Supercomputer	2004	64 kB/64 kB	1 MB	—
IBM POWER6	PC/server	2007	64 kB/64 kB	4 MB	32 MB
IBM z10	Mainframe	2008	64 kB/128 kB	3 MB	24–48 MB
Intel Core i7 EE 990	Workstation/ server	2011	6 × 32 kB/ 32 kB	1.5 MB	12 MB
IBM zEnterprise 196	Mainframe/ server	2011	24 × 64 kB/ 128 kB	24 × 1.5 MB	24 MB L3 192 MB L4

- Esta tabla muestra algunos de los tamaños de la memoria caché de diferentes sistemas.
 - El procesador Intel Core i7 990 (Extreme Edition) posee 6 *cores* o núcleos, con 3 niveles de caché:
 - En el nivel 1 de caché, por cada uno de los 6 *cores*, hay una caché de instrucciones de 32 kB y otra caché de datos de 32 kB. La de instrucciones es asociativa por conjuntos de 4 vías, mientras que la de datos es asociativa por conjunto de 8 vías.
 - En el nivel 2, por cada uno de los 6 *cores*, hay una caché de 256 kB de tipo asociativa por conjuntos de 8 vías, lo que da un total de 1,5 MB. Estas cachés son compartidas para datos e instrucciones.
 - En el nivel 3, hay un total de 12 MB compartidos entre todos los *cores*. El tipo de caché es asociativa por conjuntos de 16 vías. Esta caché es compartida para datos e instrucciones.
 - El gran computador IBM zEnterprise 196, por ejemplo, dispone de 4 *cores* o núcleos, con 4 niveles de caché:
 - En el nivel 1 de caché, cada uno de los 4 *cores* dispone de 6 cachés privadas de 64 kB para instrucciones y otras 6 de 128 kB para datos. Todas las escrituras de datos del nivel 1 se realizan también en el nivel 2 (política “*write-through*” o “*store-through*”, ver más adelante).
 - El nivel 2 de caché también se compone de 6 cachés privadas para cada uno de los 4 *cores*, cada una con 1,5 MB. Su política de escritura también es “*write-through*”.
 - El nivel 3 de caché es compartido por los 4 *cores* y tiene una capacidad de 24 MB, con una política de escritura de “*write-back*” (o “*store-in*”). Estos tres niveles de caché son “*on-chip*” (están en el chip del procesador).
 - Por último, el nivel 4 de caché también es compartido por los 4 *cores*, con un tamaño total de 192 MB. Al igual que el nivel anterior, usa una escritura de tipo “*write-back*”. Sin embargo, es “*off-chip*” al ubicarse en un chip externo al procesador.
 - Más info en: <http://www.redbooks.ibm.com/redbooks/pdfs/sg247833.pdf>

Tabla extraída del libro William Stallings, Computer Organization and Architecture, 9.ª edición, pág. 126, 2013.

ÍNDICE

- Introducción a la jerarquía de memoria
 - Introducción
 - Tecnologías de memoria
 - Parámetros del rendimiento
- Memoria caché
 - Introducción
 - Elementos de diseño
 - **Medición del rendimiento**

MEDICIÓN DEL RENDIMIENTO

- **Tiempo medio de acceso a memoria** (*average memory access time*, AMAT):

$$\begin{aligned} \text{AMAT} &= \\ &= \text{tasa de aciertos} \times \text{tiempo de acierto} + \text{tasa de fallos} \times \text{tiempo total de fallo} = \\ &= \text{tiempo de acierto} + \text{tasa de fallos} \times \text{penalización por fallo} \end{aligned}$$



$$\text{tiempo total de fallo} = \text{tiempo de acierto} + \text{penalización por fallo}$$

48

- El tiempo medio de acceso a memoria (AMAT) se calcula básicamente mediante la fórmula de la media ponderada del tiempo de acceso a memoria, en la que se utilizan como pesos la tasa de acierto y la tasa de fallos (que suman 100 % entre ambas, o 1 en tanto por 1).
- El tiempo total de fallo realmente es la suma del tiempo de acierto (primero hay que buscar si el dato está en la caché) más la penalización por fallo (el tiempo adicional que necesitamos para buscar el dato en la memoria principal y traerlo a la caché y al procesador). Es decir:

$$\text{Tiempo total de fallo} = \text{tiempo de acierto} + \text{penalización de fallo}$$

- Luego:

$$\begin{aligned} \text{AMAT} &= \text{tasa de aciertos} \times \text{tiempo de acierto} + \text{tasa de fallos} \times \text{tiempo total de fallo} = \\ &= (1 - \text{tasa de fallos}) \times \text{tiempo de acierto} + \text{tasa de fallos} \times (\text{tiempo de acierto} + \text{penalización de fallo}) = \\ &= \text{tiempo de acierto} - \text{tasa de fallos} \times \text{tiempo de acierto} + \text{tasa de fallos} \times \text{tiempo de acierto} + \\ &\quad \text{tasa de fallos} \times \text{penalización de fallo} = \\ &= \text{tiempo de acierto} + \text{tasa de fallos} \times \text{penalización de fallo} \end{aligned}$$

- Para el caso de una memoria caché multinivel (L1 y L2), la fórmula del AMAT sería:

$$\text{AMAT} = \text{tiempo de acierto}_{L1} + \text{tasa de fallos}_{L1} \times \text{penalización de fallo}_{L1}$$

- Donde:

$$\text{Penalización de fallo}_{L1} = \text{tiempo de acierto}_{L2} + \text{tasa de fallos}_{L2} \times \text{penalización de fallo}_{L2}$$

- Luego:

$$\text{AMAT} = \text{tiempo de acierto}_{L1} + \text{tasa de fallos}_{L1} \times (\text{tiempo de acierto}_{L2} + \text{tasa de fallos}_{L2} \times \text{penalización de fallo}_{L2})$$